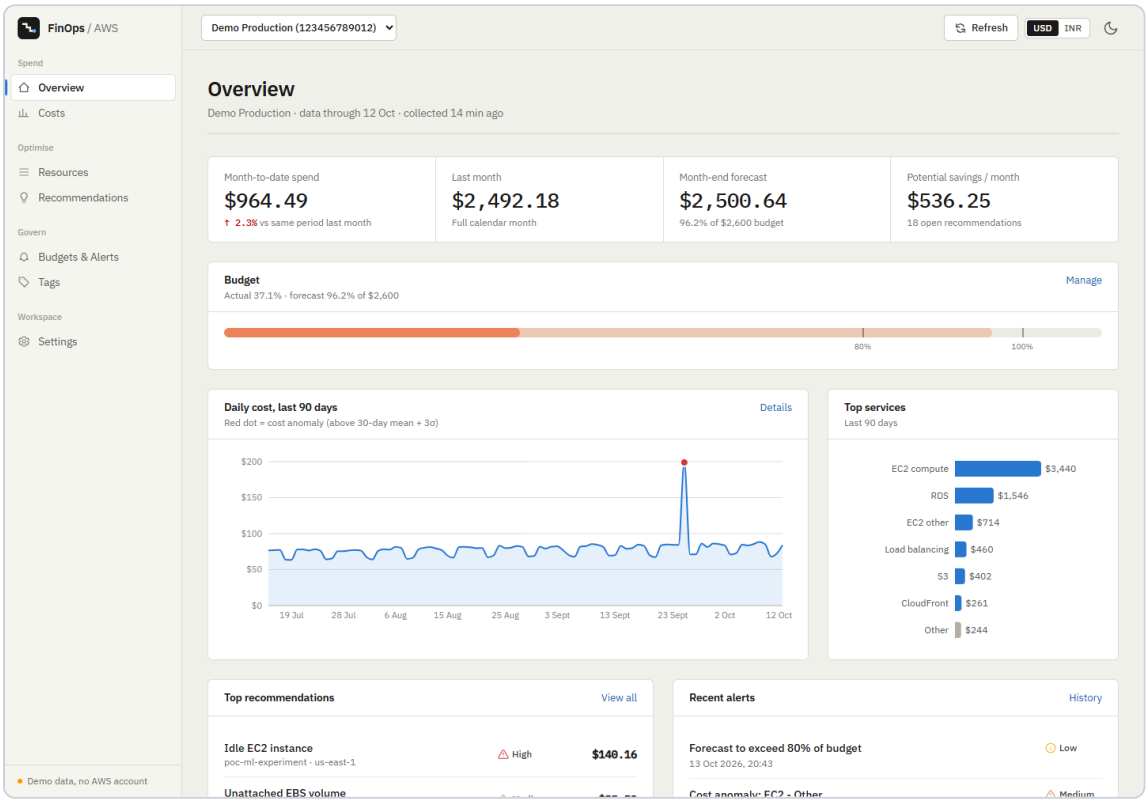
FINAL-YEAR PROJECT REPORT

Cloud-Native FinOps Platform

AWS resource monitoring and cost optimisation

A serverless web platform that connects to an AWS account through a read-only IAM role, collects cost and resource data every day, detects waste, estimates savings, forecasts the monthly bill, and alerts on budgets and cost anomalies.

8 waste-detection rules	**13** AWS services used	**~5** paid Cost Explorer calls per account per day	**220+** automated tests, 97% backend coverage



The Overview dashboard, running locally in demo mode

Stack: Python 3.12 · FastAPI · AWS Lambda · DynamoDB · AWS CDK · React 19 + TypeScript · Tailwind CSS
Region: ap-south-1 (Mumbai) · Report generated 13 October 2026

Contents

1. Introduction and problem statement
2. Objectives and features
3. System architecture
4. Backend design
5. Data collection pipeline
6. Controlling Cost Explorer spend
7. Data model (DynamoDB single table)
8. Waste-detection rules
9. Analytics: forecast, anomalies, budgets, tags
10. Alerts and notifications
11. Security and IAM design
12. Safe remediation
13. Frontend dashboard
14. REST API
15. Infrastructure as code and deployment
16. Testing and continuous integration
17. Key design decisions
18. Running the project
19. Limitations and future work

List of diagrams

- Figure 1: Deployed system architecture on AWS
- Figure 2: Backend package layering
- Figure 3: Daily collection pipeline
- Figure 4: Manual Refresh with an atomic once-per-hour guard
- Figure 5: DynamoDB single-table key design
- Figure 6: Rule engine life cycle of a finding
- Figure 7: Alert evaluation and de-duplication
- Figure 8: Cross-account access with ExternalId
- Figure 9: The four remediation safety gates
- Figure 10: Frontend structure
- Figure 11: CI pipeline

1 Introduction and problem statement

FinOps (cloud financial operations) is the practice of making cloud spending visible, accountable and efficient. On AWS, costs are spread across many services and regions, billing data arrives a day late, and the most common sources of waste are invisible in the console unless someone looks for them: instances that sit idle, disks that are no longer attached to anything, Elastic IPs that are reserved but unused, old snapshots, and S3 buckets with no lifecycle policy.

The AWS tools that address this (Cost Explorer, Budgets, Trusted Advisor, Compute Optimizer) are separate consoles, some features need a paid support plan, and Cost Explorer's API charges **$0.01 per request**, so a naive dashboard that queries it on every page load becomes expensive quickly.

> **Problem.** Small teams and students need one place that shows where the money goes, what is wasted and how much could be saved, forecasts the bill, and warns them before a budget is exceeded. It must be safe to connect (no access keys, read-only by default) and cheap to run.

This project is a cloud-native platform that does this. It runs entirely on serverless AWS services, connects to customer accounts only through a read-only IAM role, and can also run fully offline in a **demo mode** with realistic generated data, so it can be developed, tested and demonstrated without an AWS account.

2 Objectives and features

Objectives

1. Collect daily cost data and a resource inventory from an AWS account securely, without storing credentials.
2. Detect wasted resources with explainable rules and estimate the monthly saving of fixing each one.
3. Forecast the month-end bill and detect unusual cost spikes.
4. Alert by email and Slack when budgets are at risk or anomalies occur, without spamming.
5. Keep the platform's own running cost low, especially paid Cost Explorer calls.
6. Provide a clear, accessible web dashboard.
7. Define all infrastructure as code, with automated tests and CI.

Feature summary

Area	What the platform does
Account onboarding	Generates a unique ExternalId and a CloudFormation template; the customer creates a read-only role; the platform tests the connection (free) before collecting.
Cost visibility	Month-to-date, last month, month-on-month change, 90-day daily trend, breakdown by service, region and tag; USD/INR toggle.
Inventory	EC2, EBS volumes, snapshots, Elastic IPs, RDS, S3 buckets and Lambda functions across all enabled regions, each with an estimated monthly cost; searchable, filterable, CSV export.
Recommendations	8 waste rules with severities, configurable thresholds and savings estimates; findings can be dismissed and are tracked over time.
Forecasting	Linear regression on recent daily cost (spike days excluded), shown next to Cost Explorer's own forecast and confidence interval.
Anomalies	Days above mean + 3σ of the trailing 30 days, overall and per service.
Budgets and alerts	Monthly budget with thresholds (for example 80% and 100%) on actual and forecast spend; email via SNS and Slack; de-duplicated.
Tag governance	Required tags, compliance percentage, untagged resources ranked by cost, cost per tag value.
Remediation (opt-in)	Stop idle instance, snapshot-and-delete unattached volume, release Elastic IP, behind four safety gates.

3 System architecture

The platform is fully serverless and deployed as one AWS CDK stack in ap-south-1. Figure 1 shows the deployed system. The **platform account** hosts the application; each **customer account** contains only a read-only IAM role that the platform assumes.

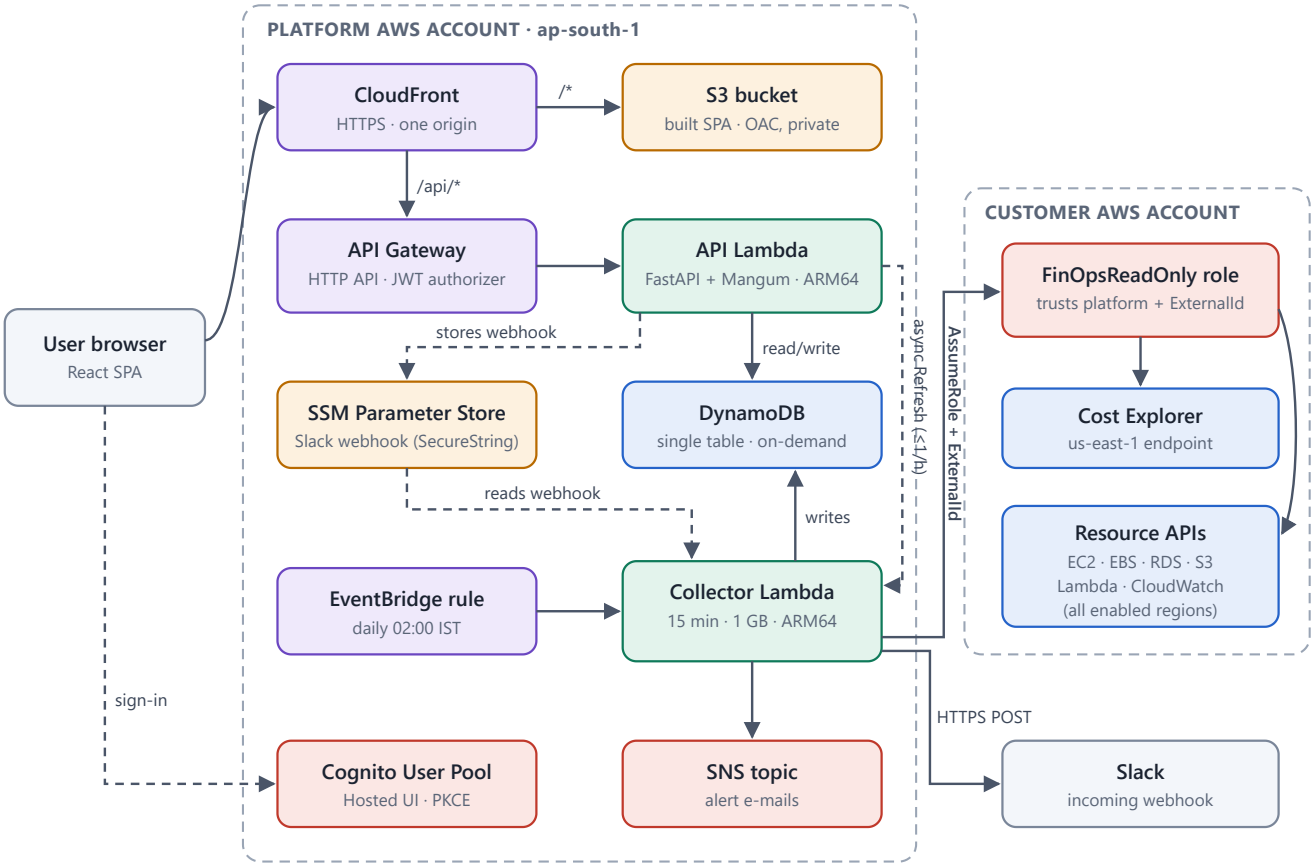


Figure 1. Deployed system architecture. Solid arrows are requests; dashed arrows are secondary flows.

How a request flows

1. **Dashboard load.** The browser loads the React app from S3 through CloudFront. Calls to `/api/*` go through the same CloudFront distribution to API Gateway, so there is one origin, no CORS, and HTTPS everywhere.
2. **Authentication.** The user signs in through the Cognito Hosted UI (authorization code + PKCE). API Gateway's JWT authorizer validates the token before the API Lambda is invoked.
3. **Reads.** The API Lambda (FastAPI wrapped by Mangum) reads only from DynamoDB. Page loads never call Cost Explorer.
4. **Collection.** Every night EventBridge triggers the collector Lambda. For each connected account it assumes the customer's read-only role, scans resources, reads metrics and costs, runs the rules and alerts, and writes everything to DynamoDB.
5. **Alerts.** E-mails go through an SNS topic; Slack messages go to a webhook whose URL is stored as an SSM SecureString.

AWS services used

Service	Role in the platform
Lambda (×2, Graviton)	API (FastAPI) and collector; ARM64 is about 20% cheaper per GB-second.
API Gateway HTTP API	REST entry point with a Cognito JWT authorizer.
DynamoDB	All application data in one table, on-demand billing, TTL for alert de-duplication keys.
EventBridge	Daily schedule (cron 20:30 UTC = 02:00 IST) with 2 retries.
S3 + CloudFront	Private bucket for the SPA, served through CloudFront with Origin Access Control.
Cognito	User pool, Hosted UI, JWTs.
SNS	E-mail alert delivery.
SSM Parameter Store	Slack webhook URLs as encrypted SecureStrings.
STS	AssumeRole into customer accounts (temporary 1-hour credentials).
Cost Explorer	Daily cost by service, region and tag; month-end forecast.
EC2, RDS, S3, Lambda, CloudWatch	Read-only inventory and utilisation metrics in the customer account.
Pricing API (optional)	Live EC2 on-demand prices instead of the static price table.
CloudFormation / CDK	Platform stack and the customer role templates.

4 Backend design

The backend is a single Python 3.12 package, `finops`, shared by the API Lambda and the collector Lambda. Both entry points run the same collection code, so there is exactly one place in the codebase that calls Cost Explorer. The package is layered so that higher layers depend only on lower ones (Figure 2).

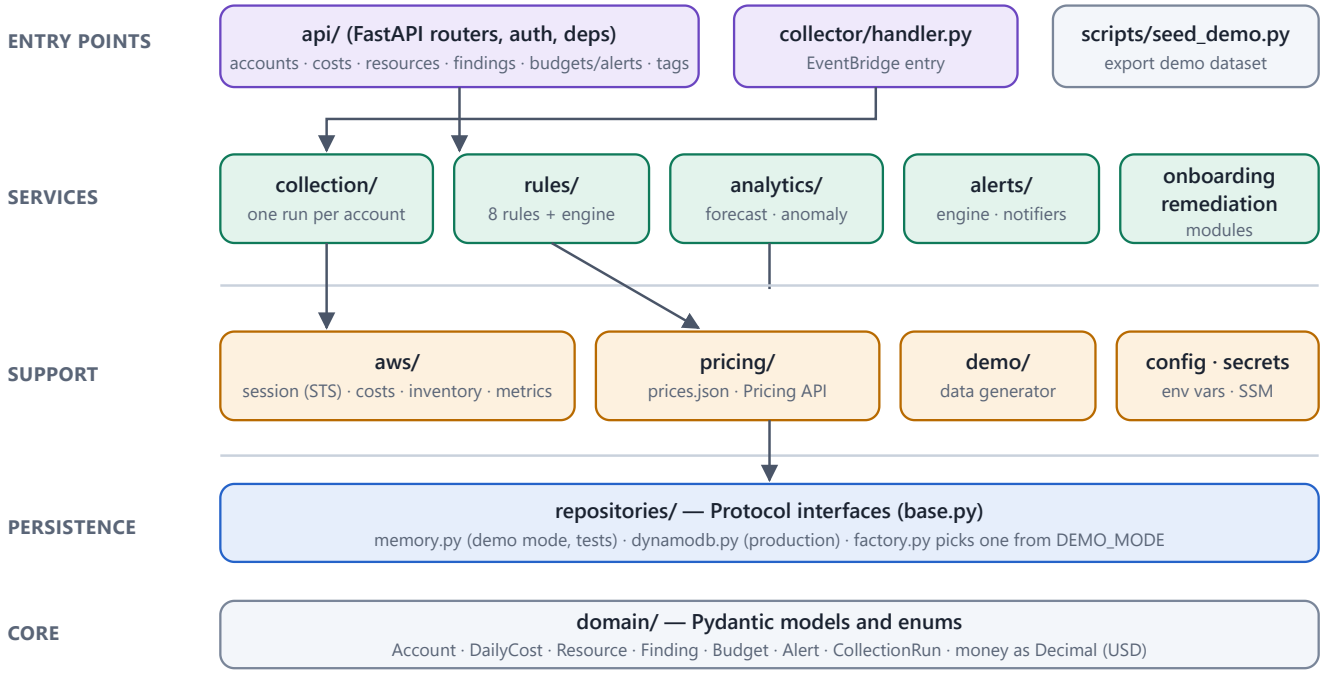


Figure 2. Backend package layering. Arrows show the main dependencies; every layer may use the domain models.

Repository pattern

Storage is accessed only through repository interfaces. Two implementations exist: an **in-memory** one used in demo mode and unit tests, and a **DynamoDB** one used in production. A shared contract test suite runs against both, proving they behave identically. This is what lets the whole application run offline with `DEMO_MODE=true`.

Money handling

All amounts are Python `Decimal` values stored in USD. DynamoDB rejects floats, and floats accumulate rounding errors when thousands of daily line items are summed. INR is only a display conversion (fixed rate 88.0, configurable).

5 Data collection pipeline

One *collection run* processes one account. It is reached from the nightly schedule and from the manual Refresh button, and it is the only code path that calls Cost Explorer.

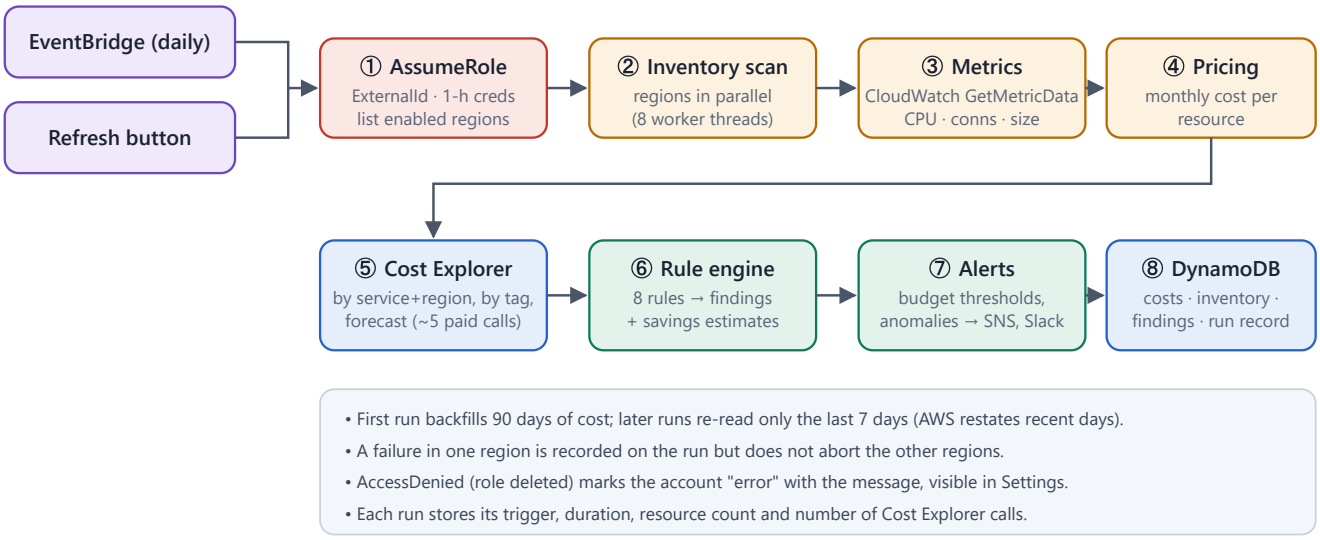


Figure 3. One collection run for one account.

What is collected

Resource	Attributes	Metric (CloudWatch, 7 days)
EC2 instance	type, state, launch/stop time, tags, attached volumes	hourly CPU average and maximum
EBS volume	type (gp2/gp3/…), size, state, attachment	–
EBS snapshot	size, age, source volume, AMI link	–
Elastic IP	association	–
RDS instance	class, engine, storage, state	DatabaseConnections maximum
S3 bucket	region, lifecycle rules, versioning, tags (metadata only)	BucketSizeBytes
Lambda function	runtime, memory, architecture	invocations

6 Controlling Cost Explorer spend

Cost Explorer's API costs $0.01 per request. The platform keeps this predictable with three measures:

- **Batching:** about five requests per account per day, grouping by two dimensions at once (service + region) and fetching the forecast once.
- **Read path isolation:** the dashboard reads only DynamoDB. A test asserts that loading every page makes zero Cost Explorer calls.
- **Manual Refresh limited to once per hour**, enforced atomically in the database (Figure 4).

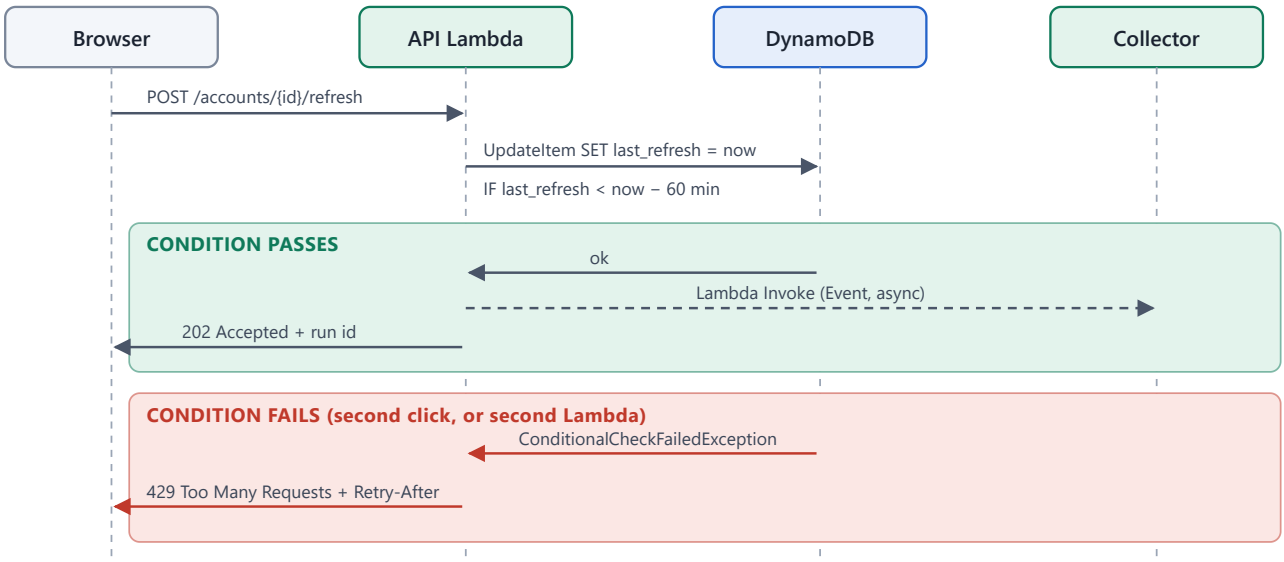


Figure 4. Manual Refresh. The check and the update are one atomic conditional write, so two simultaneous clicks cannot both pass.

The refresh runs **asynchronously** because API Gateway times out after 30 seconds while a multi-region scan can take longer. Each run, with its status and number of Cost Explorer calls, appears in the run history in Settings.

7 Data model (DynamoDB single table)

All data lives in one DynamoDB table. The partition key groups everything belonging to one account; the sort key prefix identifies the item type. Because dates are written in ISO format, a date range is a single `Query` with `SK BETWEEN`, and every dashboard view is one query.

PK (partition key)	SK (sort key)	Item type
ACCOUNT#{id} one partition per connected account	`META`	Account (+ GSI1PK=ACCOUNT to list all)
	`COST#{date}#{service}#{region}`	DailyCost
	`TAGCOST#{key}#{date}#{value}`	TagCost
	`FORECAST#{yyyy-mm}`	CostForecast (Cost Explorer)
	`RES#{type}#{region}#{resource_id}`	Resource
	`FINDING#{rule_id}#{resource_id}`	Finding (stable id)
	`BUDGET · SETTINGS`	Budget, account settings
	`ALERT#{created_at}#{id}`	Alert history
	`DEDUP#{key}`	De-dup marker (expires via TTL)
	`RUN#{started_at}#{id}`	CollectionRun

Figure 5. Single-table key design. Example: the 90-day cost chart is `PK = ACCOUNT#demo AND SK BETWEEN "COST#2026-07-15" AND "COST#2026-10-12#\uffff"`.

- **Conditional writes** implement the refresh limit and alert de-duplication atomically.
- **TTL** deletes old de-duplication markers automatically, at no cost.
- **Inventory replacement** writes the new resource set and deletes items that disappeared, so deleted resources vanish from the dashboard.
- **On-demand capacity**: the workload is a nightly burst plus light reads, so there is nothing to provision.
- **Deleting an account** is one query on its partition followed by a batch delete.

8 Waste-detection rules

Each rule is a small class with an id, a severity, default thresholds, a `detect()` method and an `estimated_monthly_savings()` method. Thresholds can be overridden per account in Settings; changing them re-evaluates findings immediately from stored data, with no AWS calls.

Rule	Sev.	Detects	Default thresholds	Estimated saving
`ec2-idle`	High	Running instance with low CPU over 7 days	avg < 5%, max < 20%, ≥ 120 hourly samples	Avg < 2%: full cost (stop). Otherwise downsize one step.
`ec2-stopped-ebs`	Med	Instance stopped > N days, volumes still billed	7 days	Cost of attached volumes
`ebs-unattached`	Med	Volume in state *available*	age ≥ 1 day	Full volume cost
`ebs-gp2-to-gp3`	Low	gp2 volume	size ≥ 1 GB	≈ 20% of volume cost
`eip-unassociated`	Low	Elastic IP not associated	–	$3.65 / month
`snapshot-old`	Low	Snapshot older than N days	90 days	size × snapshot price (upper bound)
`s3-no-lifecycle`	Low	Bucket with no enabled lifecycle rule	≥ 1 GB, IA share 40%	size × 40% × (Standard − IA)
`rds-idle`	High	DB with ≈ 0 peak connections over 7 days	max conns ≤ 0.5	Full instance cost

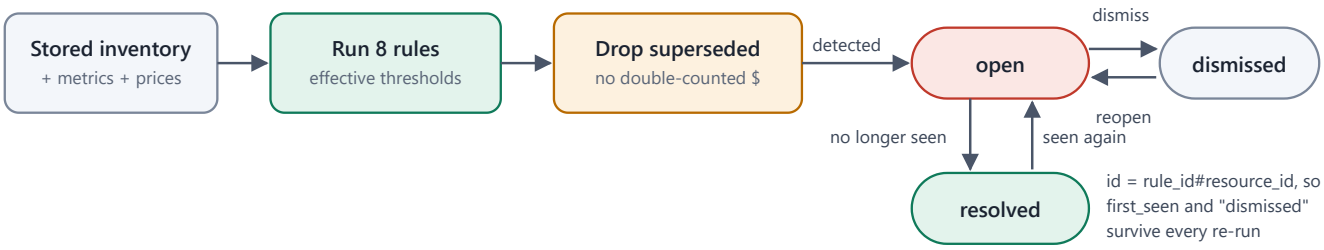


Figure 6. Rule engine and the life cycle of a finding.

Design points worth defending

- **New instances are not flagged:** `ec2-idle` needs at least 120 hourly samples (about 5 days).
- **Short peaks matter:** the rule checks maximum CPU as well as average, so a bastion host averaging 2.5% but peaking at 35% is not flagged. The demo data contains exactly this case.
- **No double counting:** a volume recommended for deletion is not also counted under gp2→gp3.
- **Honest assumptions:** without S3 access logs the platform cannot know which objects are cold, so the 40% "infrequent access" share is a visible, adjustable threshold.

- **Conservative prices:** regions missing from the price table fall back to us-east-1, usually the cheapest region.

In the demo dataset the engine produces **18 open findings worth $536.25 per month**: 3 idle EC2 instances ($221.92), 3 unattached volumes ($108.74), 1 idle RDS instance ($76.63), 3 old snapshots ($54.00), 2 stopped instances with volumes ($44.20), 2 gp2 volumes ($15.96), 2 buckets without lifecycle ($7.50) and 2 unused Elastic IPs ($7.30).

9 Analytics: forecast, anomalies, budgets, tags

Month-end forecast

The platform fits an ordinary least-squares line (numpy) to the daily cost of the trailing window, **excluding anomaly days** so that a one-off spike does not tilt the line. The forecast is

```
forecast = month-to-date + Σ (intercept + slope × day) for the remaining days
```

It is shown side by side with Cost Explorer's own machine-learning forecast and its confidence interval. The response includes slope, R^2, the number of days used and the excluded days, so the number is explainable. In the demo data: linear forecast $2,518.99 vs Cost Explorer $2,500.64 (interval $2,325.59 to $2,675.68), with one spike day excluded.

Anomaly detection

A day is an anomaly when its cost is above **mean + 3σ** of the trailing 30 days **and** at least 20% and at least $1 above the mean. The extra conditions stop pure 3σ from firing on tiny wiggles in very flat series (for example a service that costs $0.10 every day). It is computed overall and per service, so the alert names the service responsible.

Budgets

Each account has a monthly limit and thresholds (default 80% and 100%). Both **actual** and **forecast** spend are checked, so the user is warned while there is still time to act. Demo: budget $2,600, actual 37.1%, forecast 96.2%, so the "forecast to exceed 80%" alert fires.

Tag governance

The user lists required tags (for example `Owner`, `Environment`). The platform reports a compliance percentage, the count of resources missing each tag, non-compliant resources sorted by cost, and spend per tag value (untagged spend appears as `(untagged)`).

10 Alerts and notifications

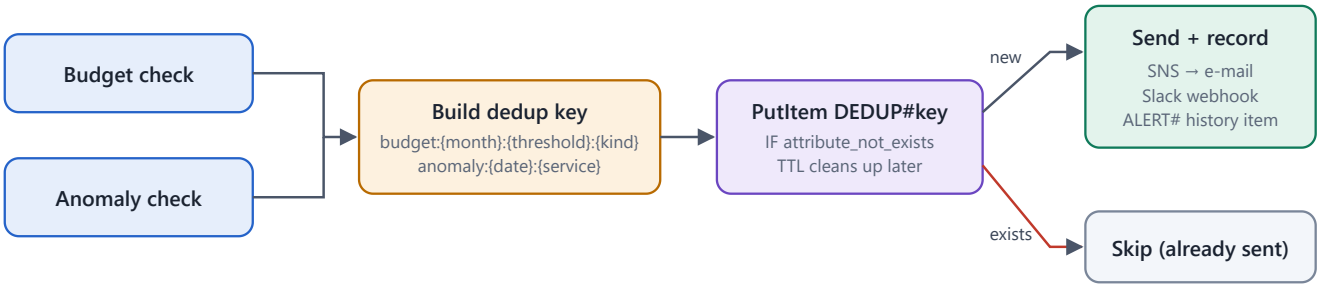


Figure 7. Alert evaluation and de-duplication.

- **E-mail:** addresses entered in Settings are subscribed to the SNS topic (AWS sends a confirmation e-mail).
- **Slack:** the webhook URL is write-only in the API. It is stored as an SSM SecureString and reads only return `slack_configured: true`.
- **Test alert:** a button sends a test message through each enabled channel without storing it.
- **History:** every alert records its severity, message and the channels it was delivered to.

11 Security and IAM design

Principles

1. **No stored credentials.** The platform never asks for access keys. It obtains temporary (1-hour) credentials with `sts:AssumeRole`; they live only in memory.
2. **Least privilege, read-only by default.** The customer role can read costs, describe resources and read metrics, nothing else. It has no `s3:GetObject`, so the platform cannot read customer data.
3. **ExternalId against the confused-deputy problem** (Figure 8).
4. **Writes are separate and opt-in** (section 12).

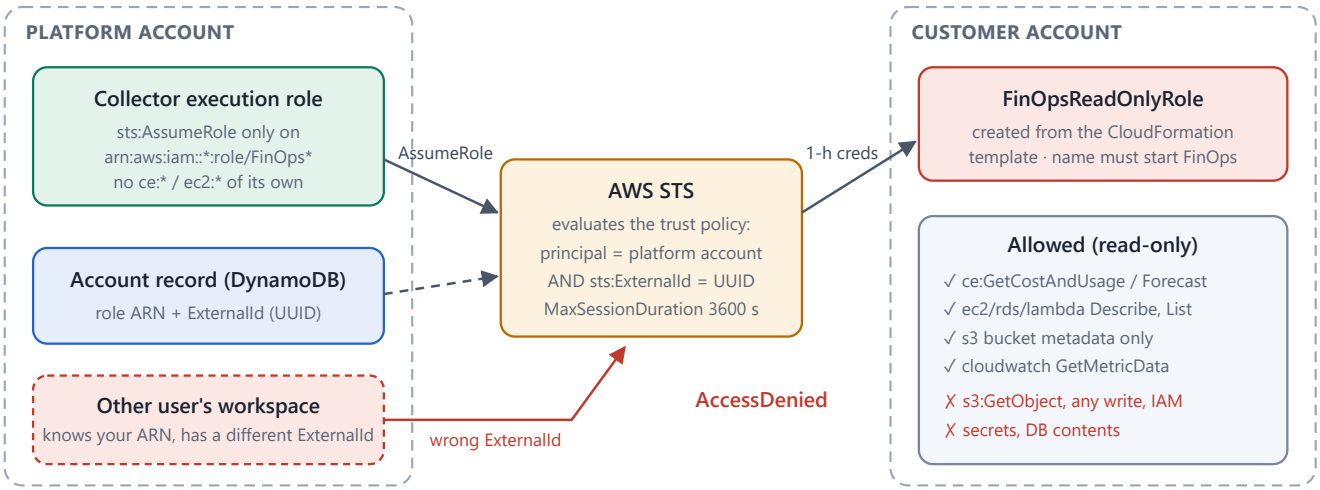


Figure 8. Cross-account access. Even someone who learns your role ARN cannot use the platform to read your account, because their workspace sends a different ExternalId.

Defence in depth

Control	Effect
Role name prefix `FinOps*`	Even a compromised platform cannot assume arbitrary roles such as `OrganizationAccountAccessRole`.
Platform roles have no `ce:*` / `ec2:*`	All reads go through the customer's role. A CDK test asserts this.
Cognito JWT authorizer	Unauthenticated requests never reach the Lambda.
Private S3 bucket + OAC	The SPA is only reachable through CloudFront.
SSM SecureString	Slack webhooks encrypted with KMS, never returned by the API.
Input validation (Pydantic)	Unknown threshold keys, malformed ARNs and bad role names are rejected with 422.
Revocation	The customer deletes the role stack; the next run fails with AccessDenied and the account is marked as error.

12 Safe remediation

Three findings can be fixed from the dashboard: stop an idle instance, snapshot and then delete an unattached volume, and release an unused Elastic IP. Because these are write actions, they pass four independent gates.

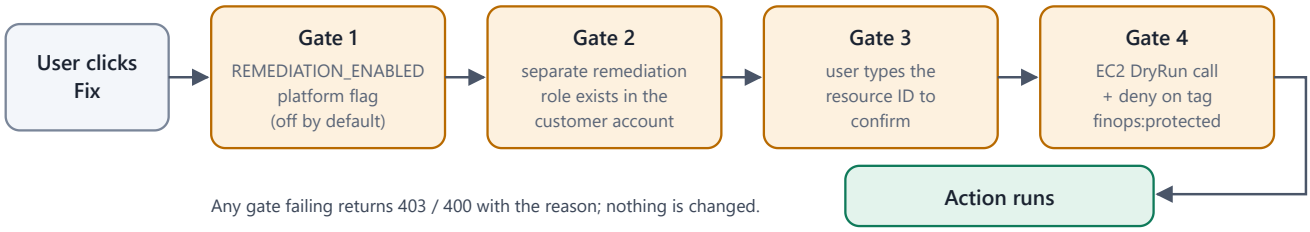


Figure 9. The four remediation safety gates. In demo mode the action is simulated.

Deleting the remediation role's CloudFormation stack revokes all write access immediately. Before any volume is deleted, a safety snapshot is taken.

13 Frontend dashboard

The dashboard is a React 19 + TypeScript single-page app built with Vite, styled with Tailwind CSS v4, with charts in Recharts and routing with React Router. It supports USD/INR, light and dark themes, phone-width layouts and accessible charts (a fixed, colour-blind-validated palette; every chart has a legend, tooltips and a table view).

The visual design is deliberately quiet, like a finance tool rather than a marketing page: warm neutral panels with hairline borders, one accent colour shared with the charts, IBM Plex Sans for text and IBM Plex Mono for money and counts, so that columns of figures line up like a ledger. Summary figures sit in one divided strip, and the sidebar groups pages by task: Spend, Optimise, Govern and Workspace.

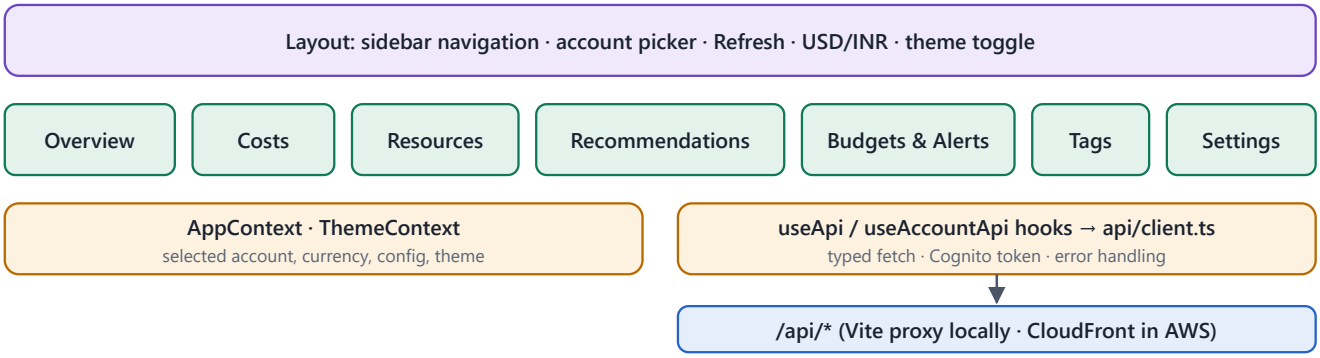
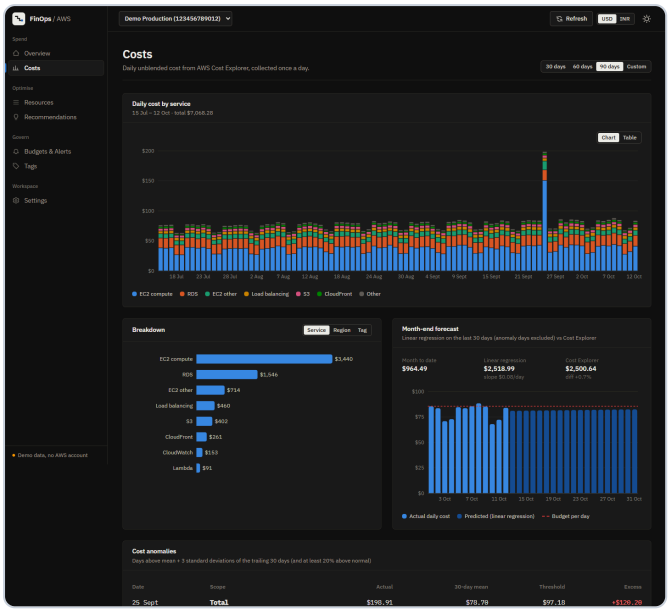


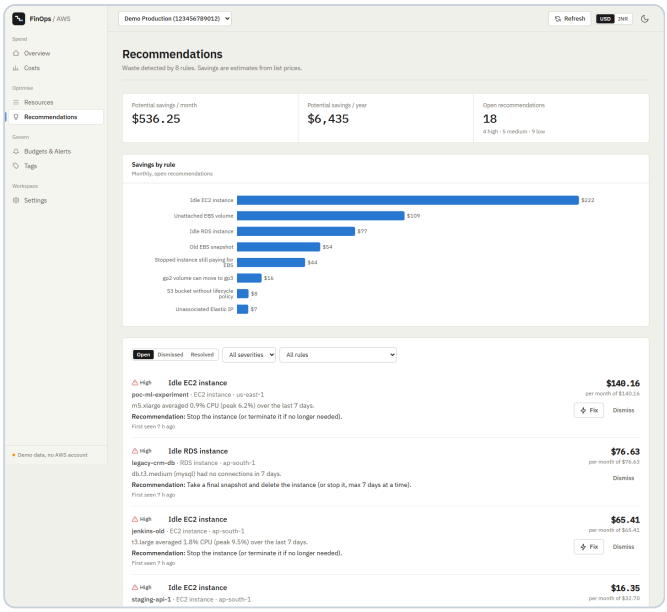
Figure 10. Frontend structure.

Page	Purpose
Overview	KPIs (month-to-date, last month, forecast, potential savings), budget bar, 90-day trend with anomaly markers, top services, top recommendations, recent alerts.
Costs	Stacked daily cost by service, breakdown by service/region/tag, forecast chart with both forecasts, anomaly list.
Resources	Inventory table with search (including `key=value` tags), filters, sorting and CSV export.
Recommendations	Findings by severity and savings, explanation, dismiss, and remediation dialog.
Budgets & Alerts	Edit budget and thresholds, alert history with delivery channels.
Tags	Compliance, missing tags, untagged resources by cost, cost per tag value.
Settings	Required tags, rule thresholds, e-mail and Slack, test alert, account connection and run history.
Connect account	Onboarding wizard: ExternalId, template download / quick-create link, CLI command, connection test.

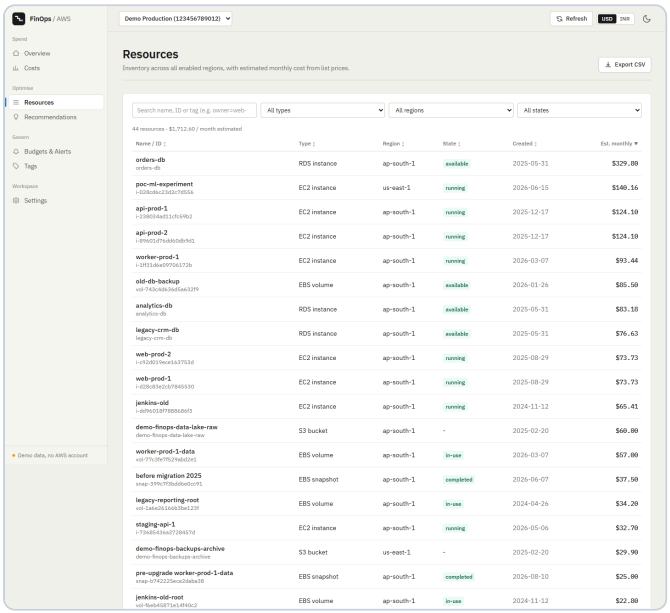
Screenshots



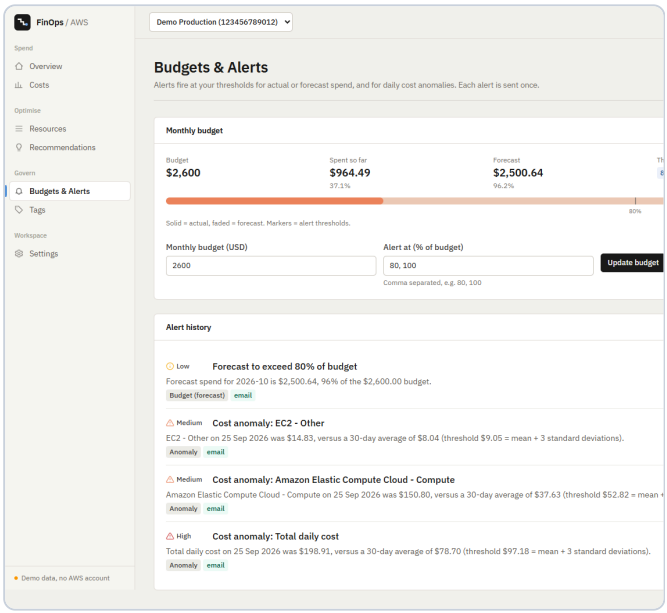
Costs (dark theme)



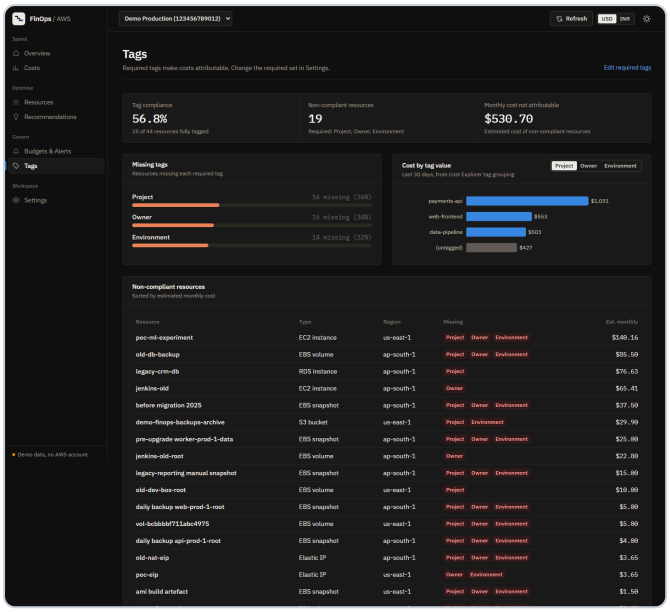
Recommendations



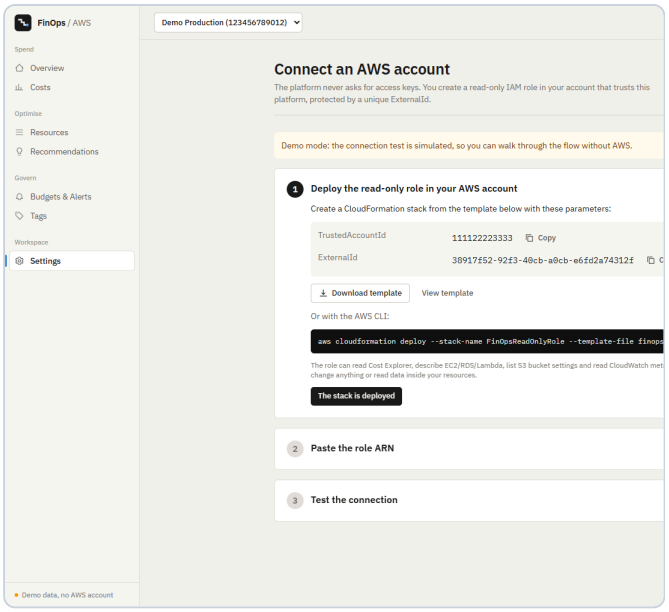
Resources



Budgets & Alerts



Tags (dark theme)



Connect account

14 REST API

The API is built with FastAPI, so it has automatic validation and interactive OpenAPI documentation at `/docs`. Account-scoped routes live under `/api/accounts/{id}/…`. The main endpoints:

Method	Path	Description
GET	`/api/health`, `/api/config`	Liveness; frontend bootstrap (public, no secrets).
GET	`/api/onboarding`	Platform account ID, fresh ExternalId, templates, CLI command.
POST	`/api/accounts`	Connect an account (role name must start with FinOps).
POST	`/api/accounts/{id}/test`	AssumeRole and list regions. Free.
POST	`/api/accounts/{id}/refresh`	202 with a run, or 429 + Retry-After inside the cooldown.
GET	`…/costs/summary`	KPIs: MTD, last month, MoM %, forecast, budget %, savings, anomalies.
GET	`…/costs/daily`, `…/costs/breakdown`	Zero-filled daily series; totals by service, region or tag.
GET	`…/costs/forecast`, `…/costs/anomalies`	Linear vs Cost Explorer forecast; anomaly days.
GET	`…/resources`, `…/resources/export.csv`	Filtered, sorted inventory; CSV.
GET/PATCH	`…/findings`, `…/findings/{fid}`	List; dismiss or reopen.
GET/POST	`…/remediation/{fid}`	Preview; execute with typed confirmation.
GET/PUT	`…/budget`, `…/settings`	Budget and thresholds; tags, rule thresholds, channels.
GET/POST	`…/alerts`, `…/alerts/test`	History; send a test alert.
GET	`…/tags/compliance`, `…/tags/costs`	Tag governance views.

15 Infrastructure as code and deployment

All infrastructure is defined in one AWS CDK (Python) stack, `FinOpsPlatformStack`. One stack deploys and rolls back atomically, with no cross-stack export lock-in. It creates:

- DynamoDB table (on-demand, TTL, point-in-time recovery; deletion protection in production) with a GSI for listing accounts;
- API Lambda and collector Lambda (Python 3.12, ARM64), built by `infra/scripts/build_lambda.py`;
- HTTP API with a Cognito JWT authorizer; Cognito user pool, app client and Hosted UI domain;
- EventBridge rule (02:00 IST, 2 retries, 2-hour max event age);
- SNS topic; IAM policies scoped to the table, the topic, the `/finops/slack-webhook/*` parameters and `role/FinOps*`;
- Private S3 bucket and CloudFront distribution (OAC, SPA rewrite function, `/api/*` behaviour, price class including India).

Customers deploy `infra/customer-templates/finops-readonly-role.yaml` (and optionally the remediation role) in their own account.

```
python infra/scripts/build_lambda.py
cd frontend && npm run build && cd ../infra
npx cdk bootstrap && npx cdk diff && npx cdk deploy
```

16 Testing and continuous integration

The project has more than 220 automated tests with about 97% backend line coverage.

Technique	Used for
moto	Mocks STS, EC2, RDS, S3, Lambda, CloudWatch, DynamoDB, SNS and SSM so AWS code runs against realistic fake services.
botocore Stubber	Cost Explorer and the Pricing API (moto lacks `GetCostForecast`); every stubbed request is validated against the real API definition.
Contract tests	The same repository tests run against the in-memory and DynamoDB implementations.
API tests	FastAPI TestClient over every route, including auth and security behaviour (write-only webhook, 429 cooldown, validation).
Cost guard test	Loads every dashboard endpoint and asserts zero Cost Explorer calls.
CDK assertion tests	Synthesise the stack and assert resources and IAM, e.g. the platform never gets `ce:*` or `ec2:*`.
Frontend checks	oxlint, TypeScript type-check and production build.

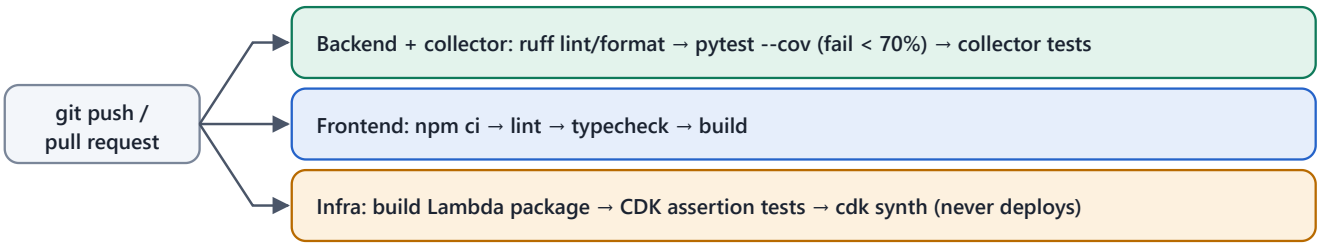


Figure 11. GitHub Actions CI: three parallel jobs.

17 Key design decisions

Decision	Reason
Serverless (Lambda, DynamoDB, API Gateway)	Near-zero idle cost; the workload is a nightly batch plus occasional dashboard reads.
One shared Python package for API and collector	Schedule and Refresh run the same code path; only one place calls Cost Explorer.
Repository pattern with in-memory and DynamoDB implementations	Offline demo mode and fast tests; contract tests prove equivalence.
DynamoDB single table, `ACCOUNT#id` / `TYPE#...`	Each dashboard view is one Query; ISO dates make range queries trivial.
Conditional writes for refresh limit and alert de-dup	Atomic: two clicks or two Lambdas can't both pass. TTL cleans up.
Money as Decimal in USD	DynamoDB rejects floats; floats accumulate rounding errors.
Asynchronous manual refresh	API Gateway's 30 s timeout vs a long multi-region scan.
Linear regression with anomaly days excluded	Explainable, cheap, robust to spikes; compared with Cost Explorer's ML forecast.
Anomaly = mean + 3σ and ≥ 20% / ≥ $1 above normal	Pure 3σ fires on tiny wiggles in flat series.
Static price table + optional Pricing API	Deterministic, offline, easy to update; live prices when wanted.
AssumeRole + ExternalId, no access keys	No long-lived secrets; prevents the confused-deputy problem.
CloudFront serves both SPA and `/api/*`	Same origin: no CORS, one URL, HTTPS everywhere.
Lambda on ARM64 (Graviton)	About 20% cheaper per GB-second.
Colour-blind-validated chart palette	Meaning never depends on colour alone.

18 Running the project

Demo mode needs no AWS account. Prerequisites: Python 3.12 and Node.js 22+.

```
# 1. Python environment (repository root)
python -m venv .venv
.venv\Scripts\activate
pip install -e "backend[dev]"
copy .env.example .env          # DEMO_MODE=true by default

# 2. API on http://localhost:8000  (Swagger UI at /docs)
cd backend
uvicorn finops.api.main:app --reload

# 3. Dashboard on http://localhost:5173  (second terminal; /api is proxied to :8000)
cd frontend
npm install
npm run dev
```

On start-up the API generates 90 days of realistic data for a demo account ("Demo Production", 44 resources, a planted cost spike), with dates relative to today so the demo always looks current.

Verified run (13 October 2026)

Check	Result
`GET /api/health`	`{"status":"ok","demo_mode":true,"region":"ap-south-1"}`
Month-to-date / last month	$964.49 / $2,492.18 (+2.3% vs same period)
Forecast	$2,500.64 (Cost Explorer), $2,518.99 (linear); 96.2% of a $2,600 budget
Recommendations	18 open, $536.25 per month potential savings
Dashboard (http://localhost:5174)	Loads through the Vite proxy with live API data (cover image is a capture of this run)
Backend + collector test suites	All passed; 97% backend coverage (CI threshold 70%)

Suggested demonstration

1. Overview: point out the spike marked in red and the forecast against the budget.
2. Recommendations: open an idle EC2 finding and explain the CPU average/maximum logic.
3. Press **Refresh** twice: the second press is rejected with a retry time (Cost Explorer guard).
4. Toggle USD/INR and the dark theme; show the table view of a chart.
5. Settings: change a rule threshold and watch findings update instantly.
6. Optional: set `REMEDIATION_ENABLED=true` to show the confirmation dialog (simulated in demo mode).

19 Limitations and future work

Limitations

- Cost data is daily and up to a day late, as in Cost Explorer itself; there is no real-time view.
- Savings use on-demand list prices; Savings Plans, Reserved Instances and enterprise discounts are not reflected.
- The S3 lifecycle saving relies on an assumed share of cold data (40% by default).
- The collector processes accounts sequentially inside one Lambda; very large organisations would exceed 15 minutes.
- Workspaces are single-organisation: all signed-in users see all connected accounts.

Future work

- SQS fan-out with one collector invocation per account for many accounts.
- Savings Plan and Reserved Instance coverage and purchase recommendations.
- More rules: idle NAT gateways, idle load balancers, over-provisioned Lambda memory.
- Multi-tenant workspaces with per-tenant isolation.
- Cost and Usage Report (CUR) ingestion via Athena for line-item detail.

> **Summary.** The platform turns raw AWS billing and inventory data into a small number of actionable, explained recommendations, while staying safe to connect (read-only, no keys, ExternalId), cheap to run (serverless, ~5 paid API calls per account per day) and fully testable offline.